

Introduction to Vite

npm

npm (Node Package Manager)

Two parts:

1. **Registry (npmjs.com)** - Central server storing millions of JavaScript packages
2. **CLI Tool** - Command line tool to download and manage packages

What npm does:

```
npm install react
```

- Connects to npmjs.com
- Downloads `react` package
- Saves it in `node_modules/` folder
- Updates `package.json` to track dependency

Common npm commands:

Command	What it does
<code>npm init -y</code>	Creates <code>package.json</code>
<code>npm install <package></code>	Downloads and saves a package
<code>npm install</code>	Downloads all packages from <code>package.json</code>
<code>npm install -D <package></code>	Install as dev dependency
<code>npm run dev</code>	Runs script from <code>package.json</code>
<code>npm run build</code>	Creates production build

Setting up React with Vite

Quick Setup (Automated)

```
npm create vite@latest  
# Select: React  
# Select: JavaScript  
cd your-project-name  
npm install  
npm run dev
```

Manual Setup (From Scratch)

Step 1: Create folder and initialize

```
mkdir react-app  
cd react-app  
npm init -y
```

Step 2: Install dependencies

```
# Runtime dependencies (your app needs these)  
npm install react react-dom  
  
# Dev dependencies (build tools)  
npm install -D vite @vitejs/plugin-react
```

Step 3: Create Vite config

Create `vite.config.js`:

```
import { defineConfig } from 'vite'  
import react from '@vitejs/plugin-react'  
  
export default defineConfig({
```

```
    plugins: [react()]
  })
```

Step 4: Create folder structure

```
mkdir src
```

Step 5: Create `index.html` (in root folder)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>React App</title>
</head>
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

Step 6: Create `src/main.jsx`

```
import { createRoot } from 'react-dom/client'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(<App />)
```

Step 7: Create `src/App.jsx`

```
function App() {
  return <h1>Hello from React!</h1>
}
```

```
export default App
```

Step 8: Add scripts to `package.json`

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

Step 9: Run

```
npm run dev
```

Understanding Build Tools

The Problem Bundlers Solve

Your project:

```
src/
├─ main.jsx
├─ App.jsx
├─ Header.jsx
├─ Footer.jsx
├─ utils.js
├─ api.js
├─ App.css
└─ Header.css
```

Each file imports others. Browser would need to:

- Load index.html

- Find main.jsx, make network request
- Parse it, find imports, make more requests
- 10 files = 10 network requests = SLOW

Bundler solution:

Combines all files into one (or few):

```
dist/
├─ index.html
├─ assets/
│   └─ index-a1b2c3.js  (ONE file with everything)
```

Browser makes 1 request, gets everything.

What Vite Does

Feature	Purpose
Dev Server	Local server at localhost:5173
JSX Transformation	Converts <code><h1></code> to JavaScript
Hot Module Replacement	Instant updates without refresh
CSS Handling	Process and inject CSS
Asset Handling	Handle images, fonts, etc.
Minification	Compress code for production
Code Splitting	Load only what's needed

CDN vs Bundler Comparison

	CDN Approach	Vite
Files	All in one file	Multiple organized files
JSX transform	Browser (slow)	Build time (fast)
Network requests	Multiple	One/few
Dev experience	Manual refresh	Instant HMR

	CDN Approach	Vite
Production ready	No	Yes

Vite Configuration

Basic Config

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()]
})
```

Advanced Config Options

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],

  // Custom dev server settings
  server: {
    port: 3000,
    open: true // Auto-open browser
  },

  // Custom build settings
  build: {
    outDir: 'build', // Output folder
    sourcemap: true
  },
})
```

```
// Path aliases
resolve: {
  alias: {
    '@': '/src',
    '@components': '/src/components'
  }
}
})
```

Plugins Explained

What is a Plugin?

A plugin is **extra functionality added to Vite**.

Think of it like smartphone apps:

- Phone by default: Calls, SMS
- Install Zomato → Can order food
- Install YouTube → Can watch videos

Similarly:

- Vite by default: Handles JS, CSS, HTML
- Install React plugin → Can handle JSX

Why Plugins Exist

Vite stays framework-agnostic. Different frameworks need different handling:

```
// For React
plugins: [react()]

// For Vue
plugins: [vue()]
```

```
// For Svelte
plugins: [svelte()]
```

One Vite, multiple frameworks via plugins.

What @vitejs/plugin-react Does

Three main jobs:

1. Configure esbuild for automatic JSX runtime

Without plugin:

```
import React from 'react' // ← Manual import needed

function App() {
  return <h1>Hello</h1>
}
```

With plugin:

```
// No React import needed

function App() {
  return <h1>Hello</h1>
}
```

2. Add Fast Refresh (via Babel)

Preserves component state during hot reloads.

3. React-specific optimizations

Development features and performance tweaks.

Plugin as Code

A plugin is just a JavaScript object with functions:

```
// Simplified plugin structure
const reactPlugin = {
  name: 'react',

  // Called for each file
  transform(code, filename) {
    if (filename.endsWith('.jsx')) {
      // Transform JSX to JavaScript
      return transformedCode;
    }
    return code;
  }
}
```

Vite calls these functions at the right time to extend behavior.

Import/Export System

Named Exports

Exporting:

```
// math.js
export const PI = 3.14;

export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}
```

Importing (must match names):

```
import { PI, add } from './math.js';

console.log(PI);           // 3.14
console.log(add(2, 3));    // 5
```

Default Export

Exporting (one per file):

```
// Calculator.jsx
function Calculator() {
  return <div>Calculator</div>;
}

export default Calculator;
```

Importing (any name works):

```
import Calculator from './Calculator.jsx'; // ✓
import Calc from './Calculator.jsx';       // ✓
import Whatever from './Calculator.jsx';    // ✓
```

Combining Both

```
// utils.js
export default function mainFunction() {}
export const helper1 = () => {};
export const helper2 = () => {};
```

```
import mainFunction, { helper1, helper2 } from './utils.js';
```

Path Rules

```
// No ./ → looks in node_modules
import { something } from 'react'

// With ./ → looks in your project
import App from './App.jsx'
```

Renaming Named Exports

```
import { PI as myPI, add as sum } from './math.js';

console.log(myPI);      // 3.14
console.log(sum(2,3)); // 5
```

Package.json Explained

Basic Structure

```
{
  "name": "my-react-app",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "vite": "^6.0.0",
    "@vitejs/plugin-react": "^4.0.0"
  }
}
```

```
}  
}
```

Dependencies vs DevDependencies

Type	Purpose	Example
<code>dependencies</code>	Needed to run the app	react, react-dom
<code>devDependencies</code>	Only needed during development	vite, eslint

Scripts Explained

```
"scripts": {  
  "dev": "vite",  
  "build": "vite build",  
  "lint": "eslint .",  
  "preview": "vite preview"  
}
```

Command	What it does	When to use
<code>npm run dev</code>	Start dev server	While coding
<code>npm run build</code>	Create production files	Before deployment
<code>npm run lint</code>	Check code issues	Anytime
<code>npm run preview</code>	Preview production build	After build

Version Symbols

```
"react": "^19.0.0"
```

Version format: `MAJOR.MINOR.PATCH`

Symbol	Example	Allows
<code>^</code> (caret)	<code>^19.0.0</code>	<code>19.x.x</code> (minor + patch updates)

Symbol	Example	Allows
<code>~</code> (tilde)	<code>~19.0.0</code>	<code>19.0.x</code> (patch updates only)
none	<code>19.0.0</code>	Exact version only

Version changes:

```

19.0.0 → 19.0.1  (Patch: bug fixes)
19.0.0 → 19.1.0  (Minor: new features, backward compatible)
19.0.0 → 20.0.0  (Major: breaking changes)

```

Exports Field

Defines which files from a package can be imported:

```

{
  "name": "some-package",
  "exports": {
    ".": "./index.js",
    "./utils": "./utils.js"
  }
}

```

Now only these work:

```

import pkg from 'some-package'           // ✓
import utils from 'some-package/utils'    // ✓
import secret from 'some-package/secret'  // ✗ Not exported

```

Purpose:

- Encapsulation (hide internal files)
- Prevent breaking changes
- Multiple entry points

JSX File Extensions

Rules for Your Vite Setup

File has JSX → **MUST** use `.jsx` extension

```
// App.jsx ✓  
function App() {  
  return <h1>Hello</h1>  
}
```

```
// App.js ✗ Will give error  
function App() {  
  return <h1>Hello</h1>  
}
```

File has only plain JavaScript → **Use** `.js`

```
// utils.js ✓  
export function calculateTotal(price, tax) {  
  return price + tax;  
}
```

Why .jsx Extension Matters

1. **Vite knows to process it** - `.jsx` triggers JSX transformation
2. **Clear intent** - Anyone knows this file has JSX
3. **Editor support** - Better syntax highlighting
4. **Convention** - Industry standard

Project Organization

```
src/  
├─ utils.js           // Pure JavaScript
```

```
|— api.js           // API calls
|— App.jsx          // React component
|— Header.jsx       // React component
|— Button.jsx       // React component
```

JSX Transformation (esbuild vs Babel)

Who Does What

Tool	Job	Why
esbuild	JSX → JavaScript transformation	Fast (100x faster than Babel)
Babel	Inject Fast Refresh tracking code	esbuild can't do this

The Actual Flow

```
Your JSX
  ↓
esbuild transforms: <h1>Hello</h1> → _jsx('h1', { children:
'Hello' })
  ↓
Babel injects Fast Refresh code (dev only)
  ↓
Browser receives optimized JavaScript
```

99% of work: esbuild (fast JSX transformation)

1% of work: Babel (Fast Refresh only)

Classic vs Automatic JSX Transform

Classic (old way):

```
import React from 'react' // ← Required

function App() {
```

```
    return <h1>Hello</h1>
  }

  // Transforms to:
  React.createElement('h1', null, 'Hello')
```

Automatic (new way):

```
// No React import needed

function App() {
  return <h1>Hello</h1>
}

// Transforms to:
import { jsx as _jsx } from 'react/jsx-runtime'
_jsx('h1', { children: 'Hello' })
```

The React plugin configures esbuild to use automatic transform.

Where is _jsx defined?

Inside React package:

```
node_modules/react/jsx-runtime.js
```

The `_jsx` function creates Virtual DOM objects (same job as `React.createElement`).

HMR (Hot Module Replacement)

What is HMR?

Update ONLY the changed file without reloading entire page.

Without HMR:

Edit file → Save → Full page reload → All state lost

With HMR:

Edit file → Save → Only that module updates → State preserved

HMR vs Fast Refresh

Term	What it is
HMR	Generic technology (works for CSS, JS, images, etc.)
Fast Refresh	React-specific HMR that preserves component state

HMR for CSS

```
/* styles.css */  
.button { color: blue; }
```

Change to red → HMR updates instantly, no reload.

Fast Refresh for React

```
function Counter() {  
  const [count, setCount] = useState(10);  
  return <button>{count}</button>;  
}
```

User clicks to 10. You change button text → Fast Refresh preserves `count = 10`.

What Babel Adds for Fast Refresh

Your code:

```
function Counter() {  
  const [count, setCount] = useState(0);
```

```
    return <p>{count}</p>;  
  }
```

Babel injects (simplified):

```
import { register, createSignature } from 'react-refresh/runtime';  
  
function Counter() {  
  const [count, setCount] = useState(0);  
  return <p>{count}</p>;  
}  
  
// Track this component  
let _signature = createSignature();  
_signature.setCustomHooks([useState]);  
register(Counter, "Counter_abc123");  
  
// Accept hot updates  
if (import.meta.hot) {  
  import.meta.hot.accept((newModule) => {  
    // Preserve state during updates  
  });  
}
```

What this does:

1. Tracks which hooks the component uses
2. When file changes, compares old vs new hooks
3. If hooks same → Preserve state
4. If hooks changed → Full remount

Fast Refresh is Development Only

	Development	Production
Fast Refresh	✓ Enabled	✗ Removed
Babel injection	✓ Yes	✗ No
Bundle size	Larger	Smaller

Run `npm run build` → Fast Refresh code is stripped out.

ESLint Configuration

What is ESLint?

A tool that analyzes code for bugs **without running it**.

Example bugs it catches:

```
const x = 5; // ← "x is assigned but never used"

if (age = 30) {} // ← "Expected === instead of ="

function greet(name) {
  const msg = `Hello ${name}`;
  const timestamp = Date.now();
  return msg; // ← "timestamp calculated but never used"
}
```

Basic Config Structure

```
export default [
  {
    files: ['**/*.js,jsx'],

    rules: {
      'no-unused-vars': 'warn', // Warn about unused variables
      'no-undef': 'error',      // Error on undefined variable
    }
  }
]
```

```

rules:
  'eqeqeq': 'error',          // Error if using == inst
  'no-console': 'off'        // Allow console.log
}
}
]

```

Severity Levels

Level	Meaning
'off' or 0	Disabled
'warn' or 1	Warning (yellow)
'error' or 2	Error (red, blocks build)

Plugins Add More Rules

```

import react from 'eslint-plugin-react'
import reactHooks from 'eslint-plugin-react-hooks'

export default [
  {
    plugins: {
      react,
      'react-hooks': reactHooks
    },
    rules: {
      'react/prop-types': 'warn',
      'react-hooks/rules-of-hooks': 'error',
      'react-hooks/exhaustive-deps': 'warn'
    }
  }
]

```

```
}  
]
```

When ESLint Runs

Option 1: Manual

```
npm run lint
```

Option 2: Editor (VS Code with ESLint extension)

Shows squiggly lines in real-time.

Option 3: Pre-commit (optional setup)

Blocks commits if errors exist.

Do You Need It?

Not required for learning. Your code works without it.

Useful for:

- Catching bugs early
- Maintaining code quality
- Team projects

StrictMode

What is StrictMode?

A development-only component that helps find bugs.

```
import { StrictMode } from 'react'  
import { createRoot } from 'react-dom/client'  
import App from './App.jsx'  
  
createRoot(document.getElementById('root')).render(  
  <App />  
)
```

```
<StrictMode>
  <App />
</StrictMode>
)
```

What It Does

Intentionally runs components twice on initial mount to expose bugs.

Example: External Variable Modification

Bad code:

```
let count = 0; // External variable

function App() {
  count = count + 1; // Modifying during render
  return <h1>Visitor #{count}</h1>;
}
```

Without StrictMode:

- Renders once
- Shows "Visitor #1" ✓

With StrictMode:

- Renders twice
- Shows "Visitor #2" ✗ (Bug exposed!)

Correct way:

```
function App() {
  const [count, setCount] = useState(1);
  return <h1>Visitor #{count}</h1>;
}
```

Example: Side Effect During Render

Bad code:

```
function App() {
  document.title = "My App"; // Side effect during render
  return <h1>Hello</h1>;
}
```

Correct way:

```
function App() {
  useEffect(() => {
    document.title = "My App";
  }, []);

  return <h1>Hello</h1>;
}
```

StrictMode Behavior

Situation	Renders
Initial mount	2x (double render)
Re-renders after that	1x (normal)
Production build	1x (StrictMode disabled)

Should You Use It?

Yes. Keep it in your code.

- Only runs in development
- No performance impact in production
- Helps write better React code
- Catches bugs early

Complete File Structure

```
my-react-app/
├─ node_modules/           # Installed packages (don't commit)
├─ src/
│   ├─ main.jsx             # Entry point
│   ├─ App.jsx              # Root component
│   └─ App.css              # Styles
├─ index.html               # HTML template
├─ package.json             # Dependencies and scripts
├─ package-lock.json        # Exact dependency versions
├─ vite.config.js           # Vite configuration
├─ eslint.config.js         # ESLint rules (optional)
└─ .gitignore               # Git ignore file
```

Key Takeaways

1. **npm** = Package manager, **npx** = Package runner
2. **Vite** = Modern build tool (fast dev server + bundler)
3. **Plugins** = Extend Vite capabilities (React plugin for JSX)
4. **esbuild** = Fast JSX transformation
5. **Babel** = Adds Fast Refresh (state preservation during HMR)
6. **JSX files must have `.jsx` extension** in your setup
7. **HMR** = Generic hot reload, **Fast Refresh** = React-specific HMR
8. **StrictMode** = Development helper (double-renders to find bugs)
9. **ESLint** = Code quality checker (optional but useful)

